How-To Library

Informatica

Configuring Cloudera Connector Powered by Teradata for Sqoop Mappings

© Copyright Informatica LLC 2018, 2021. Informatica, the Informatica logo, and Big Data Management are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https:// www.informatica.com/ trademarks.html.

Abstract

When you use Sqoop with Big Data Management® to read data from or write data to Teradata, you can configure Teradata Connector for Hadoop (TDCH) specialized Sqoop connectors. If you use a Cloudera cluster, you can configure Cloudera Connector Powered by Teradata. This article describes how to configure Cloudera Connector Powered by Teradata.

Supported Versions

- Big Data Management 10.2.1

Table of Contents

Overview. 2
Prerequisites. 2
Step 1. Download the JDBC Drivers for Sqoop Connectivity and Cloudera Connector Powered by Teradata. . . 3
Step 2. Configure the HADOOP_NODE_JDK_HOME Environment Variable in the Hadoop Connection. 3
Step 3. Configure Sqoop Properties in the JDBC Connection. 3
Step 4. Configure Sqoop Properties in the Mapping. 4
Step 5. Run the Mapping in the Hadoop Environment. 5
Supported Teradata Data Types with Cloudera Connector Powered by Teradata. 5

Overview

When you use Sqoop with Big Data Management to read data from or write data to Teradata, you can configure Cloudera Connector Powered by Teradata. Cloudera Connector Powered by Teradata is a specialized Sqoop plug-in that Cloudera provides for Teradata. It uses native protocols to connect to the Teradata database.

Informatica supports Cloudera Connector Powered by Teradata on the Blaze and Spark engines. When you run Sqoop mappings on the Blaze engine, you must complete certain configuration tasks to invoke the connector. When you run Sqoop mappings on the Spark engine, the Data Integration Service invokes the connector by default.

Configuration Tasks for Cloudera Connector Powered by Teradata

Perform the following tasks to configure Cloudera Connector Powered by Teradata with Sqoop:

- Download the JDBC drivers for Sqoop connectivity and Cloudera Connector Powered by Teradata.
- Configure the HADOOP_NODE_JDK_HOME environment variable in the Hadoop connection.
- Configure Sqoop properties in the JDBC connection.
- Configure Sqoop properties in the mapping.
- Run the mapping in the Hadoop environment.

Prerequisites

Before you configure Cloudera Connector Powered by Teradata, verify that you have completed all the configuration tasks for Informatica® Big Data Management. For more information about configuring Informatica Big Data Management, see the *Informatica Big Data Management Hadoop Integration Guide*.

Step 1. Download the JDBC Drivers for Sqoop Connectivity and Cloudera Connector Powered by Teradata

To configure Sqoop connectivity for Teradata and use Cloudera Connector Powered by Teradata, you must download JDBC driver .jar files.

1. Download any Type 4 JDBC driver that Teradata recommends for Sqoop connectivity.

   Note: The DataDirect JDBC drivers that Informatica ships are not licensed for Sqoop connectivity.
2. Perform the following steps to download the .jar files required for Cloudera Connector Powered by Teradata:
   a. Download the Cloudera Connector Powered by Teradata package from the Cloudera website. The package has the following naming convention: `sqoop-connector-teradata-<version>.tar.gz`
   b. Download all the .jar files in the package.
   c. Download the `terajdbc4.jar` file and `tdgssconfig.jar` file from the Teradata website.
3. Copy the .jar files to the following directory on the machine where the Data Integration Service runs: `<Informatica installation directory>\externaljdbcjars`

   At run time, the Data Integration Service copies the .jar files to the Hadoop distribution cache so that the .jar files are accessible to all nodes in the cluster.

   Note: After you run a mapping, if you add or delete a Type 4 JDBC driver .jar file required for Sqoop connectivity from the `externaljdbcjars` directory, changes take effect after you restart the Data Integration Service. If you run the mapping on the Blaze engine, changes take effect after you restart the Data Integration Service and Blaze Grid Manager.

Step 2. Configure the HADOOP_NODE_JDK_HOME Environment Variable in the Hadoop Connection

To run Sqoop mappings on the Blaze engine, you must configure the HADOOP_NODE_JDK_HOME environment variable in the Hadoop connection. The HADOOP_NODE_JDK_HOME environment variable represents the directory from which you run the cluster services and the JDK version that the cluster nodes use.

The JDK version that the Data Integration Service uses must be compatible with the JRE version on the cluster. You must use JDK version 1.7 or later.

1. Log in to Informatica Developer.
2. Open the Hadoop connection that you want to use to run the Sqoop mapping.
3. Click the **Cluster Environment Variables** tab.
4. Configure the HADOOP_NODE_JDK_HOME environment variable in the **Cluster Environment Variables** property as follows:

   `<cluster JDK home>/jdk<version>`

   For example, `HADOOP_NODE_JDK_HOME=<cluster JDK home>/jdk<version>`.

   Default is `/usr/java/default`.

Step 3. Configure Sqoop Properties in the JDBC Connection

To enable Sqoop connectivity, you must configure the JDBC connection properties.

You can enter the arguments that the Sqoop program must use to connect to the database. During a mapping run, the Data Integration Service merges the arguments that you specify with the default command that it constructs based on the JDBC connection properties. The arguments that you specify take precedence over the JDBC connection properties.

If you want to use the same driver to import metadata and run the mapping and do not want to specify any additional Sqoop arguments, you can select **Sqoop v1.x** from the **Use Sqoop Version** list and leave the **Sqoop Arguments** field empty. The Data Integration Service constructs the Sqoop command based on the JDBC connection properties that you specify.

However, if you want to use a different driver for run-time tasks or specify additional run-time Sqoop arguments, select **Sqoop v1.x** from the **Use Sqoop Version** list and specify the driver argument in the **Sqoop Arguments** field.

1. Log in to Informatica Developer.
2. Click **Window** > **Preferences**.
3. Expand **Informatica** and then click **Connections**.
4. Expand the domain in the **Available Connections** list.
5. Double-click **Database** and then click **JDBC**.
6. Open the JDBC connection that you want to use to connect to the database.
7. To enable Sqoop connectivity, select **Sqoop v1.x** from the **Use Sqoop Connector** list.

   Note: If you select **None**, the Data Integration Service does not trigger the Sqoop program to run the mapping in the Hadoop run-time environment. It uses JDBC to run the mapping.
8. In the **Sqoop Arguments** text box, enter the arguments that the Sqoop program must use to connect to the database.
9. To run the mapping on the Blaze engine with Cloudera Connector Powered by Teradata, you must configure the -Dsqoop.connection.factories argument. Use the argument to define the TDCH connection factory class that Sqoop must use. Configure the -Dsqoop.connection.factories argument as follows:

   ```
   -Dsqoop.connection.factories=com.cloudera.connector.teradata.TeradataManagerFactory
   ```

   Note: To run the mapping on the Spark engine, you do not need to configure the -Dsqoop.connection.factories argument. The Data Integration Service invokes Cloudera Connector Powered by Teradata by default.
10. Save the JDBC connection.

Step 4. Configure Sqoop Properties in the Mapping

You can specify additional Sqoop arguments and disable the Sqoop connector at the mapping level. The Sqoop arguments that you specify at the mapping level take precedence over the arguments that you specified in the JDBC connection.

1. Open the mapping that contains the data object for which you want to configure Sqoop properties.
2. Select the Read or Write transformation that is associated with the data object.
3. Click the **Advanced** tab.
4. To disable the Sqoop connector for the data object, select the **Disable Sqoop Connector** check box.
5. Perform one of the following steps:
   - To specify additional Sqoop import arguments for the data object, enter the import arguments in the **Additional Sqoop Import Arguments** text box.
   - To specify additional Sqoop export arguments for the data object, enter the export arguments in the **Additional Sqoop Export Arguments** text box.

   The Data Integration Service merges the additional Sqoop arguments that you specified in the mapping with the arguments that you specified in the JDBC connection and constructs the Sqoop command. The Data Integration Service then invokes Sqoop on a Hadoop node.

Step 5. Run the Mapping in the Hadoop Environment

To process data between relational databases and HDFS through Sqoop, you must run the mapping in a Hadoop environment.

1. Select a mapping from a project or folder from the **Object Explorer** view to open in the editor.
2. In the **Properties** view, select the **Run-time** tab.
3. Select **Hadoop** as the value for the validation environment and select the Blaze engine or Spark engine.
4. From the **Execution Environment** list, select **Hadoop**.
5. Specify a Hadoop connection to run the mapping.
6. Right-click an empty area in the editor and click **Run Mapping**.

   You can view log events for a mapping job in the **Monitoring** tab of the Administrator tool. You can view the results of the mapping job and the statistics in the YARN web user interface.

Supported Teradata Data Types with Cloudera Connector Powered by Teradata

Informatica supports the following Teradata data types when you use Cloudera Connector Powered by Teradata with Sqoop:

- Bigint
- Byteint
- Character
- Date
- Decimal
- Double Precision/Float/Real
- Integer
- Number(P,S)
- Numeric
- Smallint
- Time
- Timestamp
- Varbyte
- Varchar

Author

Anu Chandrasekharan

Acknowledgements

The author would like to acknowledge Muralidhara Reddy Gali, for his technical assistance.